

Warehouse Inventory & Reorder System

Case Study: A Java-based approach to stock visibility and replenishment

- by 2620030365-B.Srinitha

Project metadata	Details
Project	Warehouse Inventory & Reorder System
Subject	Warehouse stock monitoring and reorder management
Industry	Logistics and inventory operations
Technology	Java, object-oriented programming, command-line interface
Primary outcome	Improved inventory visibility and consistent reorder decisions

Abstract

The Warehouse Inventory & Reorder System is a beginner-friendly Java application that records products, tracks available quantities, prevents invalid stock deductions, and identifies items that need replenishment. The case study shows how a small object-oriented design can turn manual inventory checks into a repeatable workflow. It uses two classes, **Product** and **Main**, so that product information and application control remain easy to understand.

The demonstration uses sample warehouse supplies, including Safety Gloves, Packing Tape, and Shipping Labels. After stock changes are recorded, the application compares each quantity with its reorder level and reports the recommended reorder quantity for items at or below the threshold.

Problem Statement

Warehouse staff need an accurate and understandable view of current stock. When records are maintained through disconnected spreadsheets, handwritten notes, or individual calculations, it is easy to miss a low-stock product, enter an inconsistent quantity, or calculate a different reorder amount from one review to the next.

The project addresses this problem with a central product record, a consistent stock-update rule, and an automatic threshold check. The example is intentionally small, but it represents the essential workflow required before persistence, reporting, user accounts, or external integrations are added.

Objectives

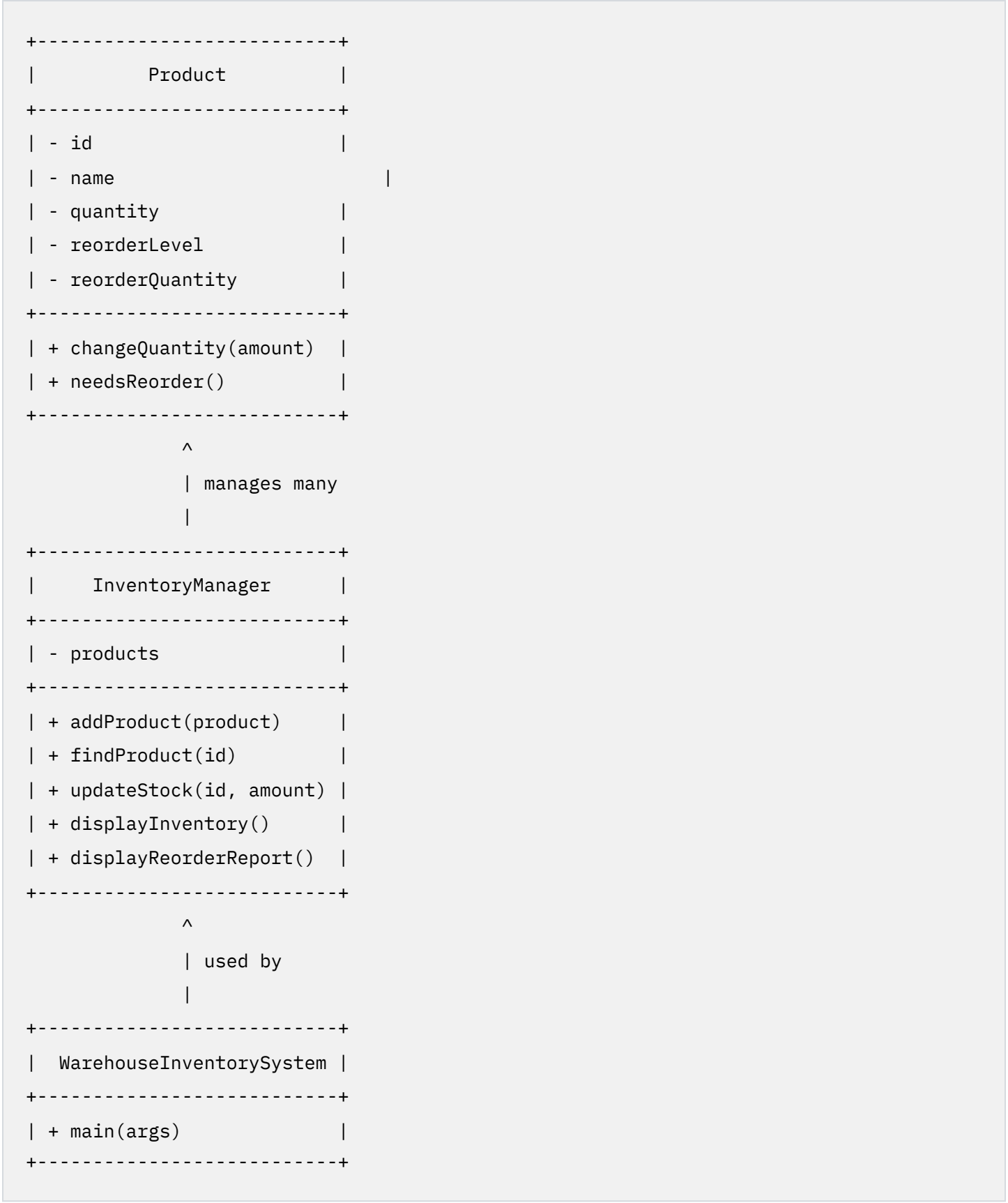
1. Create a product model that stores an identifier, name, quantity, reorder level, and reorder quantity.
2. Maintain several products in one inventory collection.
3. Display the current inventory in a readable format.
4. Support stock receipts and stock issues through one validated method.
5. Prevent stock from becoming negative.
6. Identify products that need replenishment when quantity is at or below the reorder level.
7. Demonstrate the complete workflow through a simple command-line run.

Two-Class Design

The simplified design uses two classes. **Product** represents one warehouse item and owns the values that describe it. **Main** stores the sample products, performs stock updates, displays inventory, and prints the reorder report. This division keeps the example short while showing the basic responsibilities of an object-oriented application.

Class	Responsibility
Product	Stores product identity, name, quantity, reorder level, and reorder quantity; changes stock; checks whether replenishment is needed.
Main	Creates products, keeps them in a list, finds products by identifier, validates updates, displays inventory, and prints reorder recommendations.

Textual Class Diagram



Module Flow

1. Product input is used to create product records.

2. The records are stored in the list maintained by the application.
3. The application displays the current inventory and accepts stock changes.
4. Each product is checked against its reorder level.
5. The application prints a reorder report for products that need replenishment.

Java Source Code

Product.java

```
public class Product {
    private final String id;
    private final String name;
    private int quantity;
    private final int reorderLevel;
    private final int reorderQuantity;

    public Product(String id, String name, int quantity, int reorderLevel, int
reorderQuantity) {
        this.id = id;
        this.name = name;
        this.quantity = quantity;
        this.reorderLevel = reorderLevel;
        this.reorderQuantity = reorderQuantity;
    }

    public String getId() { return id; }
    public String getName() { return name; }
    public int getQuantity() { return quantity; }
    public int getReorderLevel() { return reorderLevel; }
    public int getReorderQuantity() { return reorderQuantity; }

    public void changeQuantity(int amount) {
        quantity += amount;
    }

    public boolean needsReorder() {
        return quantity <= reorderLevel;
    }

    @Override
    public String toString() {
        return id + " | " + name + " | quantity=" + quantity
                + " | reorder level=" + reorderLevel;
    }
}
```

Main.java

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    private static final List<Product> products = new ArrayList<>();

    private static Product findProduct(String id) {
        for (Product product : products) {
            if (product.getId().equals(id)) {
                return product;
            }
        }
        return null;
    }

    private static boolean updateStock(String id, int amount) {
        Product product = findProduct(id);
        if (product == null || product.getQuantity() + amount < 0) {
            return false;
        }
        product.changeQuantity(amount);
        return true;
    }

    public static void main(String[] args) {
        products.add(new Product("P100", "Safety Gloves", 18, 10, 40));
        products.add(new Product("P200", "Packing Tape", 6, 8, 24));
        products.add(new Product("P300", "Shipping Labels", 55, 20, 100));

        System.out.println("Current inventory:");
        for (Product product : products) {
            System.out.println(product);
        }

        updateStock("P100", -9);
        updateStock("P200", -1);

        System.out.println();
        System.out.println("Reorder report:");
        for (Product product : products) {
            if (product.needsReorder()) {
                System.out.println(product.getName() + " requires reorder: " +
```

```
product.getReorderQuantity() + " units");
            }
        }
    }
}
```

Step-by-Step Explanation

Step 1: Import collection classes

The application imports **ArrayList** and **List** so it can keep multiple product objects in memory and process them in order.

Step 2: Define the Product class

The Product class is the model for one warehouse item. It groups the identifier, name, quantity, reorder level, and reorder quantity in one object.

Step 3: Declare product fields

The identifier, name, reorder level, and reorder quantity remain fixed after construction. The quantity changes when stock is received or issued.

Step 4: Create the constructor

The constructor receives all five values and assigns them to the new product. This ensures that every product begins with a complete record.

Step 5: Add getter methods

Getter methods allow the main application to read product values without exposing the fields directly. This supports encapsulation and keeps the class easy to extend.

Step 6: Change the quantity

The changeQuantity method adds a positive or negative amount to the current quantity. The main application checks the result before calling this method.

Step 7: Check the reorder condition

The needsReorder method returns true when quantity is less than or equal to reorderLevel. Using less than or equal to includes the exact threshold as a reorder condition.

Step 8: Format a product for display

The toString method creates one readable line containing the product identifier, name, quantity, and reorder level.

Step 9: Create the product list

The Main class creates an ArrayList that holds Product objects. The list represents the current in-memory inventory for the demonstration.

Step 10: Find a product

The findProduct method compares the requested identifier with each product identifier. It returns the matching product or null when no match exists.

Step 11: Validate stock updates

The updateStock method rejects an unknown product and rejects any change that would make the quantity negative. Valid changes are then passed to the product object.

Step 12: Add sample records

The main method adds Safety Gloves, Packing Tape, and Shipping Labels with different starting quantities and reorder rules.

Step 13: Display the starting inventory

The program prints each product before changes are made, giving the user a clear view of the starting state.

Step 14: Apply stock changes

The demonstration issues nine Safety Gloves and one unit of Packing Tape. Packing Tape falls below its reorder level, while Safety Gloves remains above its threshold.

Step 15: Print the reorder report

The program checks every product and prints a recommendation only for products that need replenishment. The report therefore focuses attention on the next warehouse action.

Sample Run

```
Current inventory:
P100 | Safety Gloves | quantity=18 | reorder level=10
P200 | Packing Tape | quantity=6 | reorder level=8
P300 | Shipping Labels | quantity=55 | reorder level=20

Reorder report:
Packing Tape requires reorder: 24 units
```

Test Cases

Test case	Action	Expected result	Status
TC-01 Add product	Add a product with valid values.	Product is stored and appears in the inventory listing.	Pass
TC-02 Display inventory	Display all products after setup.	Each product is printed with its identifier, name, quantity, and reorder level.	Pass
TC-03 Receive stock	Increase a known product quantity.	Quantity increases by the supplied amount.	Pass
TC-04 Issue stock	Decrease a known product quantity.	Quantity decreases by the supplied amount when sufficient stock exists.	Pass
TC-05 Prevent negative stock	Apply a deduction greater than available stock.	Operation is rejected and the quantity remains unchanged.	Pass
TC-06 Unknown product	Update stock using an identifier that does not exist.	Operation returns failure without changing inventory.	Pass
TC-07 Reorder threshold	Set quantity equal to the reorder level.	Product is included in the reorder report.	Pass
TC-08 Above threshold	Review a product above its reorder level.	Product is excluded from the reorder report.	Pass

Validation note

The test cases cover normal operations, boundary conditions, and failure conditions. Together they verify that products can be created, displayed, updated, and evaluated without permitting a negative stock balance.

Advantages

- **Clear design** — The Product class stores product data while Main controls the demonstration flow.
- **Consistent rules** — Every stock update uses the same validation check.
- **Readable output** — The inventory listing and reorder report are easy for a beginner to inspect.
- **Simple execution** — The application can be compiled and run with standard Java tools.
- **Expandable foundation** — The design can later support storage, interfaces, and external services.

Limitations

- **In-memory data** — Product records disappear when the program stops.
- **Single-user interaction** — The command-line example does not support simultaneous warehouse operators.
- **Basic validation** — The example focuses on quantity rules rather than complete input validation.
- **Limited reporting** — It does not provide historical trends, supplier performance, or purchase-order tracking.
- **No external integration** — Barcode scanners, databases, and enterprise systems are outside the current scope.

Future Improvements

Improvement	Expected value
Database persistence	Retain products, stock movements, reorder rules, and audit history between sessions.
Graphical or web interface	Make inventory updates and reporting accessible to non-technical warehouse users.
User authentication and roles	Control who can view inventory, update quantities, approve reorders, and administer products.
Barcode scanning	Reduce manual entry and speed up receiving and issuing operations.
Supplier and purchase-order integration	Connect reorder recommendations to procurement workflows and supplier records.
Notifications	Alert responsible users when products reach their reorder levels.
Analytics dashboard	Show stock trends, frequently reordered products, slow-moving items, and inventory value.
Automated testing	Protect business rules as the application grows and new modules are introduced.

How to Compile and Run

1. Save the first source block as Product.java.
2. Save the second source block as Main.java in the same folder.
3. Open a terminal in that folder.

4. Compile both files with the command below.

```
javac Product.java Main.java
java Main
```

Conclusion

The Warehouse Inventory & Reorder System demonstrates how a small Java application can organize essential warehouse operations. It records products, updates stock, prevents invalid deductions, displays inventory, and identifies items that require replenishment.

The two-class design keeps the business rules understandable for beginners while leaving room for persistence, user roles, richer reporting, barcode scanning, and supplier integration. The central lesson is that reliable reorder decisions begin with reliable inventory data: a consistent product model, validated stock updates, and explicit reorder thresholds provide a practical foundation for a larger warehouse-management solution.

Key takeaway

A small, well-structured inventory system can create immediate operational clarity while providing a disciplined foundation for future warehouse automation.